

Modul Media Programming, Lehrveranstaltung AV Programming



Prof. Dr. Eva Wilk

HAW Hamburg, Fakultät INF, Studiengang Medieninformatik

Festlegungen zu Test 1 und Test 2

- Der Test kann nicht wiederholt werden.
- Wir bieten, für Krankheitsfälle, noch einen Ersatztermin für die beiden Tests an. An einem der Tests nicht teilnimmt, benötigt ein Attest (AU-Bescheinigung), um am Ersatztermin teilnehmen zu können.
- Das Ergebnis von Test 1 und von Test 2 wird nur für die Klausur am 8.10. angerechnet. Wer diese Klausur nicht mitschreibt, verliert die Punkte aus den Tests und muss diese erneut ablegen.

Thema 4 - Übersicht

Wie werden Videosignale gespeichert, codiert und datenreduziert, und wie lässt sich das mit Python nachvollziehen?

- Video als räumlich-zeitliche Repräsentation
- Container vs. Codec
- Grundlagen der Videodatenreduktion
- H.264, H.265, AV1
- Übungsaufgaben zum 9.6.2026

Thema 4 - Lernziel

Die Studierenden unterscheiden Video-Containerformate und Videocodecs, erläutern das Grundprinzip moderner verlustbehafteter Videodatenreduktion und ordnen H.264, H.265 und AV1 hinsichtlich Grundprinzip, Effizienz und Einsatzkontext ein.

Sie untersuchen die Eigenschaften von Videodateien mit Python, ffprobe und ffmpeg.

Video als digitales Datenobjekt

- Folge von Frames über die Zeit
- zentrale Parameter:
 - Auflösung
 - Bildrate
 - Dauer
 - Pixel-Format
 - Bitrate
- Dateiendung allein reicht nicht

Video als digitales Datenobjekt

Video, aus technischer Sicht:

- zeitlich geordnete Folge von Einzelbildern

zusätzliche Eigenschaften:

- Auflösung, Bildrate und Dauer.

streambezogene Merkmale:

- Pixel-Format oder Bitrate.

Endung .mp4: zeigt nicht an wie der Videostrom codiert wurde

ffprobe: zur Analyse von Format- und Streaminformationen

Container und Codec

- Container: z. B. MP4, MKV
- Codec: z. B. H.264, H.265, AV1
- *Container* organisiert Streams
- *Codec* beschreibt Codierung/Decodierung des Videostroms
- eine Datei kann mehrere Streams enthalten

Container und Codec

Ein *Container* ist die Struktur, in der Audio-, Video- und weitere Datenströme organisiert werden.

Ein *Codec* beschreibt, wie der Videostrom codiert und decodiert wird.

Die FFmpeg-Dokumentation trennt Formate einerseits und Codecs andererseits in eigenen Dokumentationen.

Beispiel 13: Videodatei mit ffprobe untersuchen

- Python + ffprobe
- Container, Codec, Auflösung, Bildrate auslesen
- Video mit OpenCV abspielen

Beispiel 13: Videodatei mit ffprobe untersuchen

```
import json
import subprocess

video = "beispiel.mp4"
cmd = [
    "ffprobe", "-v", "error", "-print_format", "json",
    "-show_format", "-show_streams", video
]
result = subprocess.run(cmd, capture_output=True, text=True, check=True)
info = json.loads(result.stdout)

video_stream = next(s for s in info["streams"] if s["codec_type"] == "video")

print("Datei:", video)
print("Container:", info["format"]["format_name"])
print("Videocodec:", video_stream.get("codec_name"))
print("Auflösung:", video_stream.get("width"), "x", video_stream.get("height"))
print("Pixel-Format:", video_stream.get("pix_fmt"))
print("Bildrate:", video_stream.get("avg_frame_rate"))
print("Bitrate gesamt:", info["format"].get("bit_rate"))
print("Dauer:", info["format"].get("duration"))
```

Beispiel 13: Videodatei mit ffprobe untersuchen

Ausgabe:

```
Datei: video.mp4  
Container: mov,mp4,m4a,3gp,3g2,mj2  
Videocodec: av1  
Auflösung: 640 x 360  
Pixel-Format: yuv420p  
Bildrate: 30/1  
Bitrate gesamt: 836436  
Dauer: 10.000000
```

Beispiel 13: Videodatei mit ffprobe untersuchen

Die Datei ist eine MP4-/QuickTime-artige Containerdatei mit einem AV1-codierten Videostream. Das Video hat 640×360 Pixel, verwendet das Pixel-Format yuv420p, läuft mit 30 Bildern pro Sekunde, hat eine Gesamtbitrate von etwa 0,84 Mbit/s und ist 10 Sekunden lang.

Begriffe

- **Container:** mov,mp4,m4a,3gp,3g2,mj2 *Formatnamen des FFmpeg-De muxers.
Dieser Demuxer kann die eng verwandte ISO-BMFF-/QuickTime-Familie lesen,
wie hier aufgelistet. → Format des Containers ist MP4-/QuickTime-artig.*
- **Videocodec:** av1 *Der Videostream ist mit AV1 codiert*
- **Auflösung:** 640 x 360 *Jeder Frame hat 640 Pixel Breite, 360 Pixel Höhe*
- **Pixel-Format:** yuv420p ***YUV:** Luminanz-/Chrominanz-Darstellung
4:2:0: Farbunterabtastung.
Die Farbinformation wird mit geringerer räumlicher Auflösung
gespeichert als die Helligkeit.
p: **planar.** Die Kanäle liegen getrennt vor, nicht interleaved (RGB)*
- **Bildrate:** 30/1 *30 Bilder pro Sekunde*

Begriffe

- **Demuxer:** Öffnet einen Container und trennt die darin enthaltenen einzelnen Datenströme. Beispiel: holt aus einer MP4-Datei heraus a. den Video-Stream, b. den Audio-Stream, c. ggf. Untertitel, Metadaten. Der Muxer hat diese Daten vorher zusammengepackt.
- **AV1:** Neuerer Videocodec
- **YUV:** Y = Helligkeit, UV = Farbanteile. (Gibt es auch als YCbCr) Luminanz-/Chrominanz-Darstellung). Das menschliche Sehsystem ist für Helligkeit empfindlicher als für Farbe. Deshalb kann man Farbinformation oft stärker reduzieren als Helligkeitsinformation.
- **4:2:0:** Farbunterabtastung. Die Helligkeit wird voll abgetastet. Die Farbinformation hat geringere Auflösung → für mehrere Pixel wird weniger Farbinformation gespeichert als Helligkeitsinformation. Das spart Datenrate
- **p:** planar. Die Kanäle liegen getrennt im Speicher, z. B. erst alle Y-Werte, dann alle U-Werte, dann alle V-Werte.
- **interleaved:** Die Werte verschiedener Kanäle liegen abwechselnd nebeneinander, z. B. RGBRGBRGB ...

Begriffe

1. Was bedeuten Y, U, V?

Das Bild wird nicht direkt als Rot, Grün, Blau gespeichert, sondern aufgeteilt in

Y = Helligkeitsinformation und U/V = Farbinformation

Vorteil: Das menschliche Auge ist für Helligkeit empfindlicher als für feine Farbdetails, deshalb kann man die Farbinformation stärker reduzieren als die Helligkeit. Darauf basiert dann auch 4:2:0.

2. Wie hängt das mit RGB zusammen?

RGB beschreibt direkt: wie viel Rot, wie viel Grün, wie viel Blau.

YUV/YCbCr beschreibt stattdessen: wie hell etwas ist, und welche Farbe relativ dazu vorliegt

Man kann von RGB nach YUV/YCbCr umrechnen. Vereinfacht gilt:

Y ist eine gewichtete Mischung aus R, G, B. U/Cb und V/Cr beschreiben die Abweichung der Farbkanäle von dieser Helligkeitskomponente.

Also: Aus RGB wird erst eine Helligkeitskomponente gebildet, die verbleibende Farbinformation wird in zwei Kanäle ausgelagert

Begriffe

3. Wie hängen YUV und Cb/Cr zusammen?

YUV ist der ältere, historisch eher analoge Begriff. YCbCr ist die digitale Form, die in Video- und Bildcodierung praktisch wichtiger ist.

Dabei gilt ungefähr: Y bleibt die Helligkeitskomponente,

Cb entspricht ungefähr dem „blau minus Helligkeit“-Signal, Cr entspricht ungefähr dem „rot minus Helligkeit“-Signal $\rightarrow U \approx Cb, V \approx Cr$

(Nicht exakt identisch, weil YUV historisch aus analogem Fernsehen kommt, YCbCr digital normiert ist und mit skalierten Werten arbeitet)

4. Was bedeutet 4:2:0?

Die Notation 4:2:0 beschreibt, wie dicht diese Kanäle relativ zueinander abgetastet werden:

Für 4 Pixel in einem 2×2-Block gilt:

4:4:4: jedes Pixel hat eigene Farbwerte

4:2:2: je zwei Pixel pro Zeile teilen sich Farbwerte

4:2:0: je vier Pixel im 2×2-Block teilen sich Farbwerte

Videodatenreduktion

Rohvideo ist sehr groß, daher

- Reduktion *räumlicher* Redundanz
- Reduktion *zeitlicher* Redundanz
- verlustbehaftete Codierung

Ziel: Speicherung, Übertragung, Wiedergabe praktikabel machen

Beispiel 14: Rohdatenmenge und Dateigröße

- Rohdatenmenge grob abschätzen
- mit tatsächlicher Dateigröße vergleichen

Aufgabe 9

a.) Berechnen Sie näherungsweise die unkomprimierte Rohdatenmenge eines Videos mit:

10 s Dauer

640 × 360 Pixel

30 fps

Annahme: Jedes Frame liegt als RGB-Bild mit 8 Bit pro Kanal vor.

Bearbeiten Sie die Aufgabe in drei Schritten:

1. Anzahl der Frames

2. Datenmenge pro Frame

3. gesamte Rohdatenmenge in Byte und in MB

b.) Schreiben Sie einen allgemeinen Python-Code-Schnipsel, der diese Größe für beliebige Eingabewerte berechnet.

Dauer: 5 Minuten

Beispiel 14: Rohdatenmenge und Dateigröße

```
video = Path("video.mp4")

cmd = [ "ffprobe", "-v", "error", "-print_format", "json", "-show_format",
        "-show_streams", str(video) ]
result = subprocess.run(cmd, capture_output=True, text=True, check=True)
info = json.loads(result.stdout)

v = next(s for s in info["streams"] if s["codec_type"] == "video")
width = int(v["width"])
height = int(v["height"])
duration = float(info["format"]["duration"])

bytes_per_frame = width * height * 3 # grobe Annahme: RGB mit 3 Byte pro Pixel

num, den = v["avg_frame_rate"].split("/") # Bildrate aus Bruch lesen, z. B. "25/1"
fps = float(num) / float(den) if float(den) != 0 else 0
frame_count_est = duration * fps
raw_size_est = bytes_per_frame * frame_count_est
file_size = video.stat().st_size
print("Geschätzte Rohgröße (MB):", round(raw_size_est / 1_000_000, 2))
print("Tatsächliche Dateigröße (MB):", round(file_size / 1_000_000, 2))
```

Beispiel 14: Rohdatenmenge und Dateigröße

```
Datei: video.mp4  
Auflösung: 640 x 360  
Bildrate: 30.0 fps  
Dauer: 10.0 s  
Geschätzte Rohgröße (MB): 207.36  
Tatsächliche Dateigröße (MB): 1.05  
Grobe Kompressionsrelation: 198.3 : 1
```

Wie kommt die starke Kompression zustande?

Der Codec nutzt

- räumliche Vorhersage innerhalb eines Bildes
- zeitliche Vorhersage zwischen Bildern
- Bewegungskompensation
- Transform des Residuals
- Quantisierung
- Entropiekodierung
- Farbunterabtastung wie yuv420p

Bei diesem Video konkret: Die Auflösung ist mittelgroß, die Szene ist nicht sehr komplex und ändert sich über die Zeit nur sehr wenig, yuv420p wird verwendet, und die komprimierte Datei wird mit der Rohdatenmenge (unkomprimiertes Video) verglichen.

Videokompression am Beispiel AV1

1. Räumliche Vorhersage innerhalb eines Bildes (Intra-Prädiktion)

- **nutzt aus:** benachbarte Pixel innerhalb eines Bildes ähneln sich oft, z. B. in glatten Flächen, Helligkeitsverläufen oder entlang einfacher Kanten
- **Idee:** Ein Block wird aus bereits bekannten Nachbarbereichen vorhergesagt
- **Speicherung:** der Vorhersagemodus und die Abweichung zwischen Vorhersage und tatsächlichem Bildinhalt

2. Zeitliche Vorhersage zwischen Bildern (Inter-Prädiktion)

- **nutzt aus:** Bild zu Bild ändert sich wenig, besonders bei ruhiger Kamera oder langsamer Bewegung
- **Idee:** Das aktuelle Bild oder ein Block darin wird aus einem oder mehreren früheren oder späteren Bildern vorhergesagt.
- **Speicherung:** Referenzinformation und Änderung

Videokompression am Beispiel AV1

3. Bewegungskompensation

- **nutzt aus:** Änderungen zwischen Bildern sind oft keine völlig neuen Inhalte, sondern verschobene Inhalte
- **Idee:** Ein Objekt taucht nicht „neu“ auf, sondern ist häufig nur an einer anderen Position. Der Codec sucht daher passende Bereiche im Referenzbild und beschreibt die Verschiebung.
- **Speicherung:** ein Bewegungsvektor plus die Restabweichung

4. Transform des Residuals

- **nutzt aus:** der verbleibende Fehler ist nach guter Vorhersage oft strukturell einfacher als das Originalsignal
- **Idee:** Die Abweichung zwischen Vorhersage und Original wird in eine Darstellung überführt, in der sich Energie günstiger konzentriert.
- **Speicherung:** nicht der Fehler als Pixelmatrix, sondern dessen Transformkoeffizienten

Videokompression am Beispiel AV1

5. Quantisierung

- **nutzt aus:** Dass nicht jede kleine Abweichung visuell gleich wichtig ist.
- **Idee:** Kleine oder weniger relevante Transformanteile werden gröber dargestellt oder auf null gesetzt.
- **Speicherung:** eine vereinfachte, gröber quantisierte Version der Koeffizienten

6. Entropiekodierung

- **nutzt aus:** die nach Vorhersage und Quantisierung verbleibenden Daten sind **statistisch günstig verteilt**, enthalten also z. B. viele kleine Werte oder viele Nullen
- **Idee:** Häufige oder einfache Symbole bekommen kurze Bitdarstellungen, seltene längere.
- **Speicherung:** mit gleichem Informationsgehalt nur bitökonomischer

Videokompression am Beispiel AV1

7. Farbunterabtastung, z. B. yuv420p

- **nutzt aus:** das menschliche Sehsystem reagiert auf Helligkeitsdetails empfindlicher als auf feine Farbdetails.
- **Idee:** Die Helligkeit wird feiner gespeichert als die Farbe.
- **Speicherung:** viel Detail in Y, weniger Detail in Cb/Cr

Zusammengefasst: prediction, transform, quantization, entropy coding (, loop filtering)

In-Loop-Filter:

- werden auf das rekonstruierte Bild angewendet, bevor es als Referenzbild weiterverwendet wird
 - reduzieren typische Artefakte blockbasierter, quantisierter Videocodierung, z. B. Blockgrenzen und Ringing
 - sind Teil des Codecs
- Verbesserung der sichtbaren Bildqualität und der Qualität der Referenz für spätere Vorhersage

H.264, H.265, AV1 – Einordnung und Unterschiede

H.264 / AVC (Advanced Video Coding)

extrem verbreitet, robust, sehr wichtig für Kompatibilität und Bestandsökosysteme

- älterer, sehr verbreiteter Referenzcodec

H.265 / HEVC (High efficiency video coding)

effizientere Weiterentwicklung von H.264

- Nachfolger von H.264
- höhere Effizienz, meist höhere Komplexität

AV1 (AOMedia Video 1)

öffentlich zugänglicher Spezifikation und Royalty-freie Patentpolitik

- neuerer offener Codec
- ebenfalls gleicher Grundansatz
- wichtig für moderne Web-/Streaming-Kontexte

„optimiert für 4K/8K, adaptive streaming und HDR“ (geringe Bitrate, einsetzbar über viele Plattformen und Browser)

https://www.itu.int/rec/dologin_pub.asp?id=T-REC-H.264-200305-S%21%21PDF-E&lang=e&type=items
<https://www.itu.int/rec/t-rec-h.265>
<https://aomedia.org/specifications/av1/>

HDR und SDR

- SDR = Standard Dynamic Range: geringerer darstellbarer Bereich von Helligkeiten und meist geringerer Farbumfang
- HDR = High Dynamic Range: größerer darstellbarer Bereich zwischen sehr dunklen und sehr hellen Bildanteilen
 - nicht mehr Pixel, sondern: mehr Helligkeitsumfang, mehr Abstufungen, oft größerer Farbraum, höhere Bittiefe

wichtig:

Aufnahme, Codierung, Übertragung und Display müssen HDR unterstützen

Räumliche Auflösung von Videobildern

HD – Full HD – 4K – 8K

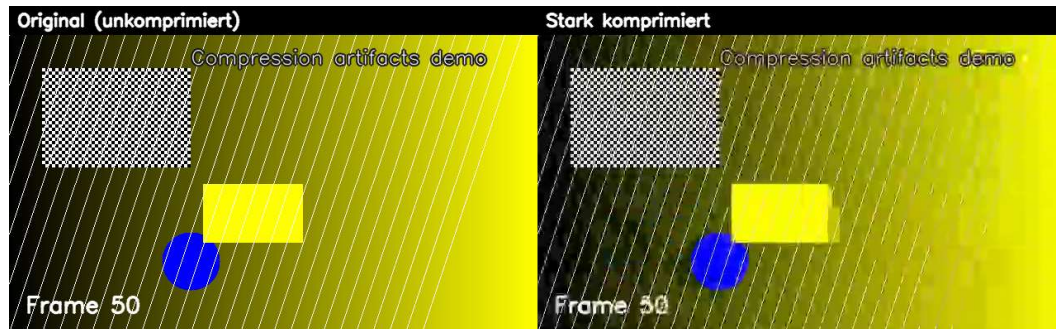
- HD: 1280 × 720 Pixel ([High Definition](#))
- Full HD: 1920 × 1080 Pixel ([Full High Definition](#))
- 4K / UHD: meist 3840 × 2160 Pixel (ursprünglich 4096 x 2160 Pixel im Kino-/Produktionsbereich, jetzt Consumer-Bereich) ([Ultra High Definition](#))
- 8K / UHD: meist 7680 × 4320 Pixel (doppelte Breite, doppelte Höhe ggü. 4K)

Beispiel 15: Demo Coding-Artefakte

- unkomprimiertes Testvideo erzeugen
- stark komprimierte Variante erzeugen
- Original und komprimierte Version vergleichen
- typische Artefakte:
 - Blockartefakte
 - Banding
 - Detailverlust
 - unsaubere Kanten

Beispiel 15: Demo Coding-Artefakte

```
....  
if __name__ == "__main__":  
    create_uncompressed_video()  
    create_compressed_video()  
    save_comparison_frame(second=2.0)  
    print_sizes()
```



Unkomprimiert:
video_demo\original_uncompressed.avi
-> 103.70 MB

Komprimiert:
video_demo\compressed_artifacts.mp4 ->
0.05 MB

Artefakte: sichtbar an

Kompressionsrelation ca.: 2093.2:1

- Farbverläufen → „Banding“
- feinen Mustern → Verlust von Details
- Texte und Kanten → Blockbildung, Unschärfe
- Bewegung → zusätzliche Kompressionsfehler

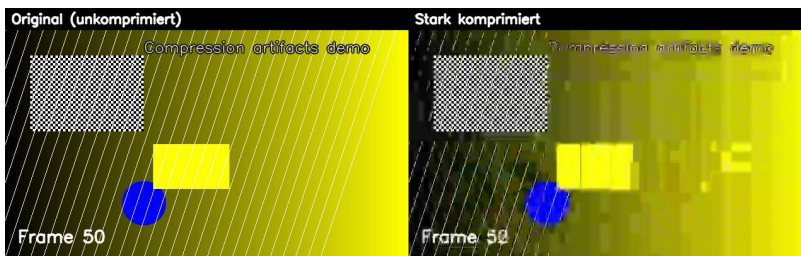
Beispiel 15: Demo Coding-Artefakte

Verstärkung der Kompression mit

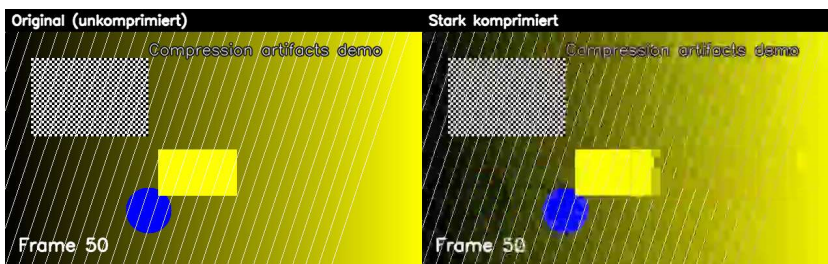
```
"-b:v", "40k",      # sehr wenig Daten pro Sekunde erlaubt (Zielbitrate: 40 kbit/s)
"-maxrate", "40k",  # auch kurzfristig darf die Bitrate nicht höher werden
"-bufsize", "20k",  # kleiner Puffer → kaum Spielraum für Bitratenschwankungen
```

oder

```
"-crf", "45",      # sehr grobe Qualitätsstufe, starke Quantisierung („45“ ist extrem hoch)
                  # kleinerer Wert = bessere Qualität, größere Datei, größerer Wert = schlechtere Qualität, kleinere Datei
```



A. Unkomprimiert:
 video_demo_A\original_uncompressed.avi
 -> 103.70 MB
 Komprimiert:
 video_demo_A\compressed_artifacts.mp4
 -> 0.03 MB
 Kompressionsrelation ca.: 4038.3:1



B. Unkomprimiert:
 video_demo_B\original_uncompressed
 .avi -> 103.70 MB
 Komprimiert:
 video_demo_B\compressed_artifacts.
 mp4 -> 0.03 MB
 Kompressionsrelation ca.: 3039.7:1

Quellen

- FFprobe: [Documentation](#)
- FFmpeg Formats: [Documentation](#)
- FFmpeg: [Documentation](#)
- OpenCV: [cv::VideoCapture Class Reference](#)
- ITU-T H.264: [Advanced video coding for generic audiovisual services](#)
- ITU-T H.265: [High efficiency video coding](#)
- Alliance for Open Media: [AV1 Video Codec](#)